C path.c

```c

#include <stdio.h>

#define N 6
#define INF 99999

char *places[N] = {
    "Main Gate",
    "Library",
    "Canteen",
    "CSE Block",
    "Auditorium",
    "Hostel"
};

int graph[N][N] = {
    {0,100,150,0,0,0},
    {100,0,0,120,200,0},
    {150,0,0,80,0,180},
    {0,120,80,0,90,0},
    {0,200,0,90,0,110},
    {0,0,180,0,110,0}
};

void shortestPath(int source, int destination)
{
    int dist[N], visited[N], parent[N];
```

path.c

```c
int graph[N][N] = {
};

void shortestPath(int source, int destination)
{
    int dist[N], visited[N], parent[N];
    int i, j, u, min, path[N], count = 0;

    for (i = 0; i < N; i++) {
        dist[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }

    dist[source] = 0;

    for (i = 0; i < N; i++) {
        min = INF;
        u = -1;

        for (j = 0; j < N; j++) {
            if (!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }

        if (u == -1)
            break;

        visited[u] = 1;

        for (j = 0; j < N; j++) {
            if (graph[u][j] > 0 && !visited[j]) {
                if (dist[u] + graph[u][j] < dist[j]) {
                    dist[j] = dist[u] + graph[u][j];
```

Ln 26, Col 2 Space

path.c

```
26  {
38      for (i = 0; i < N; i++) {
54          for (j = 0; j < N; j++) {
55              if (graph[u][j] > 0 && !visited[j]) {
56                  if (dist[u] + graph[u][j] < dist[j]) {
57                      dist[j] = dist[u] + graph[u][j];
58                      parent[j] = u;
59                  }
60              }
61          }
62      }
63
64      if (dist[destination] == INF) {
65          printf("\nNo path found.\n");
66          return;
67      }
68
69      i = destination;
70
71      while (i != -1 && count < N) {
72          path[count++] = i;
73          i = parent[i];
74      }
75
76      printf("\nShortest Path:\n");
77
78      for (i = count - 1; i >= 0; i--) {
79          printf("%s", places[path[i]]);
80          if (i > 0)
81              printf(" -> ");
82      }
83
84      printf("\nDistance: %d meters\n", dist[destination]);
85  }
86
87  int main(void)
88  {
```

C path.c

```c

int main(void)
{
    int source, destination, i;

    printf("COLLEGE SHORTEST PATH FINDER\n");
    printf("---------------------------\n");

    for (i = 0; i < N; i++)
        printf("%d. %s\n", i + 1, places[i]);

    printf("\nEnter starting location (1-6): ");

    if (scanf("%d", &source) != 1 || source < 1 || source > N) {
        printf("Invalid input!\n");
        return 1;
    }

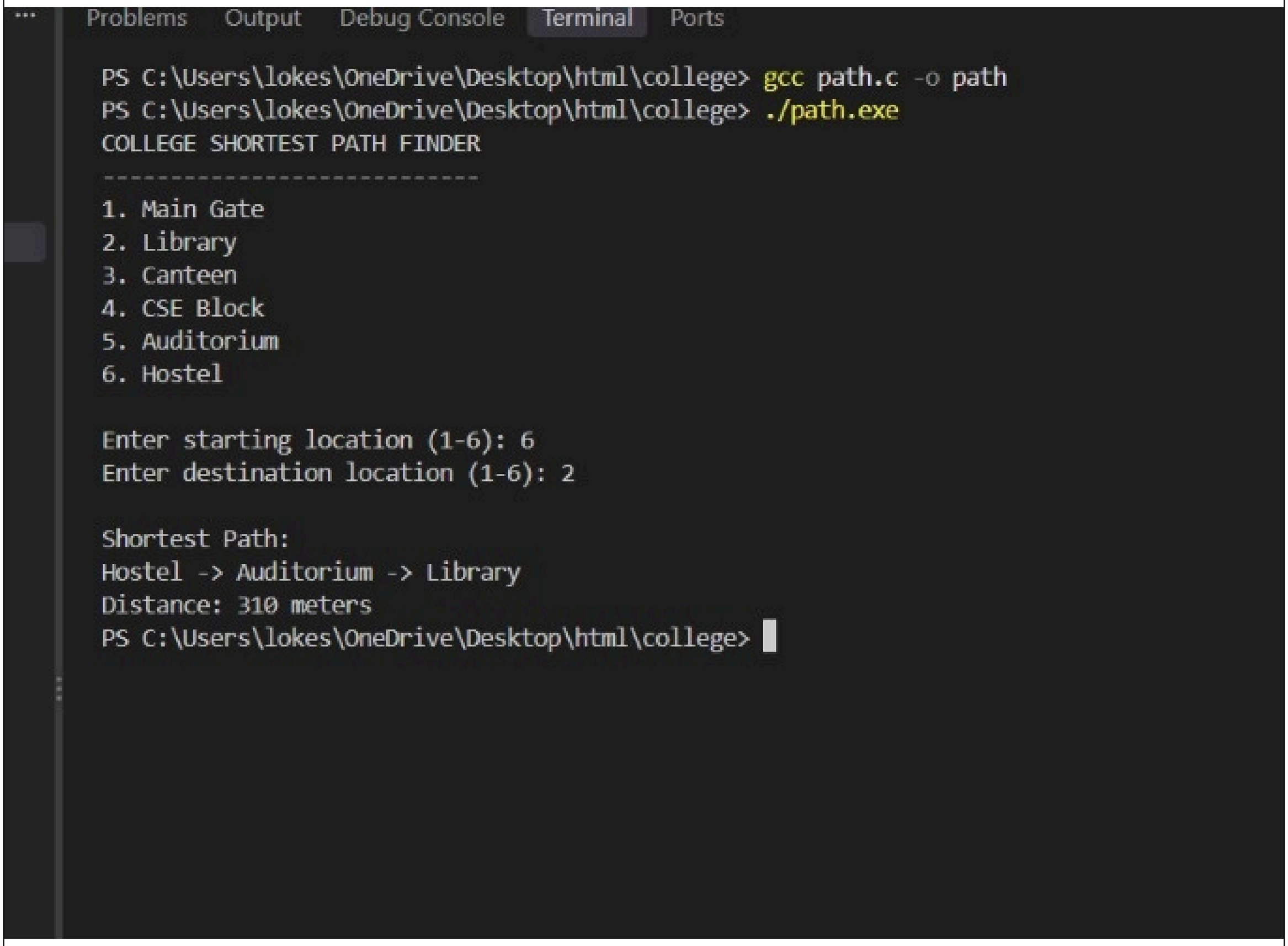
    printf("Enter destination location (1-6): ");

    if (scanf("%d", &destination) != 1 ||
        destination < 1 || destination > N) {
        printf("Invalid input!\n");
        return 1;
    }

    shortestPath(source - 1, destination - 1);

    return 0;
}


```


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\lokes\OneDrive\Desktop\html\college> gcc path.c -o path
PS C:\Users\lokes\OneDrive\Desktop\html\college> ./path.exe
COLLEGE SHORTEST PATH FINDER
1. Main Gate
2. Library
3. Canteen
4. CSE Block
5. Auditorium
6. Hostel
Enter starting location (1-6): 6
Enter destination location (1-6): 2
Shortest Path:
Hostel -> Auditorium -> Library
Distance: 310 meters
PS C:\Users\lokes\OneDrive\Desktop\html\college>

College Campus Shortest Path Finder

Abstract

A C Program to Find the Shortest Path Between Campus Locations Using Dijkstra's Algorithm

1. Introduction

Finding the shortest path between two locations in a campus is a common real-life problem. This project presents a College Campus Shortest Path Finder, implemented in C, which helps the user to find the minimum distance path between any two locations in the college using Dijkstra's algorithm. The campus is modeled as a weighted graph where each location is a node and the distance between locations is the weight of the edge.

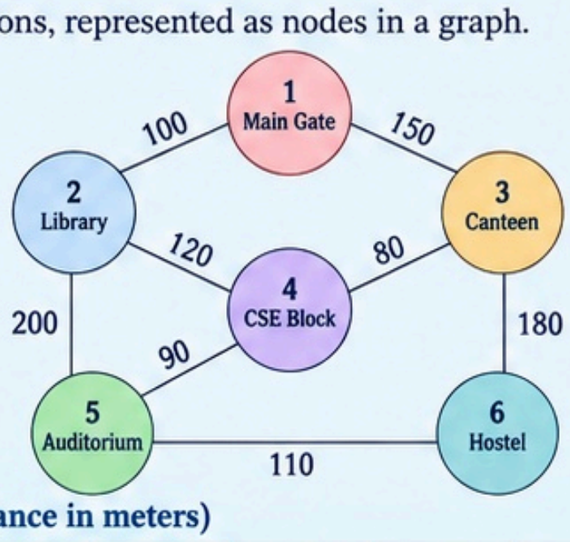
2. Objective

- To implement a program in C that finds the shortest path between two given locations in a college campus using Dijkstra's algorithm.

3. Campus Locations and Graph

The campus has 6 locations, represented as nodes in a graph.

1. Main Gate
2. Library
3. Canteen
4. CSE Block
5. Auditorium
6. Hostel



Adjacency Matrix (Distance in meters)

From/To	1	2	3	4	5	6
1	0	100	150	0	0	0
2	100	0	0	120	200	0
3	150	0	0	80	0	180
4	0	120	80	0	90	0
5	0	200	0	90	0	110
6	0	0	180	0	110	0

4. Methodology

The program uses Dijkstra's algorithm to calculate the shortest path. It maintains three arrays – distance, visited and parent. The algorithm selects the unvisited node with the minimum distance, updates the distances of its adjacent nodes, and repeats the process until all nodes are visited. Finally, the path is reconstructed using the parent array and displayed along with the total distance.

5. Key Features

- Uses Dijkstra's algorithm for shortest path calculation
- Predefined campus locations and distance matrix
- Accepts user input for source and destination
- Displays the shortest path and total distance
- Handles invalid input and no-path cases

6. Results

The program shows the shortest path between the selected locations along with the minimum distance in meters. The output is easy to understand and helps users find the best route across the campus.

7. Advantages and Limitations

Advantages

- ✓ Simple and easy to use
- ✓ Efficient for small to medium networks
- ✓ Helps in real-life navigation within the campus

Limitations

- × Works only for predefined locations
- × Does not consider real-time traffic or dynamic changes
- × Unsuitable for very large networks without modifications

8. Conclusion

The College Campus Shortest Path Finder is a simple yet effective program that demonstrates the use of Dijkstra's algorithm in solving a real-life navigation problem. It can be further extended by adding more locations, dynamic distances and a user-friendly interface.